

COMMONLY USED MATHEMATICA CODE

We make extensive use of *Mathematica*, together with the computer package *gln.m* written by Kevin A. Broughan. Our session starts with

```
SetDirectory[NotebookDirectory[]];
<< gln.m
```

The Schur polynomials can be generated by `SchurPolynomial[]`. In particular, the Dirichlet coefficient $\lambda_{\Pi}(p^n)$ for Π on $\text{PGL}_d(\mathbb{A}_{\mathbb{Q}})$ can be created by:

```
Schu[n_, d_, nota_ : a] := SchurPolynomial[Table[nota[i], {i, 1, d}], Join[
  ConstantArray[0, d - 2], {n}]]
```

The following commands will be used in extracting coefficients and common factors:

```
coeffsAndMonoms[p_, vars_] := Module[{rules, exps, coeffs, monoms}, rules =
  CoefficientRules[p, vars]; exps = Keys[rules];
  coeffs = Values[rules]; monoms = Times @@ MapThread[Power, {vars, #}] & /@
  exps; {coeffs, monoms}]

commonMonomialFactor[expr_, vars_, ass_ : True] := Module[{terms, exps, mins},
  terms = List @@ Expand[expr]; exps = Table[Exponent[t, v], {t, terms}, {v,
  vars}]; mins = Simplify[Min /@ Transpose[exps], ass]; Times @@ MapThread
  [#1^#2 &, {vars, mins}]]

pullCommonMonomial[expr_, vars_, ass_ : True] := Module[{fac}, fac =
  commonMonomialFactor[expr, vars, ass]; fac FullSimplify[expr/fac, ass]]

OmegaRule2[[Omega]_, ass_ : True] := HoldPattern[c_. a[1]^p_. a[2]^q_.] :> With
  [{s = FullSimplify[Min[p, q], ass]}, c \[Omega]^s a[1]^FullSimplify[p - s,
  ass] a[2]^ FullSimplify[q - s, ass]];

OmegaRule3[[Omega]_, ass_ : True] := HoldPattern[c_. a[1]^p_. a[2]^q_. a[3]^r_.] :> With[{s = FullSimplify[Min[p, q, r], ass]}, c \[Omega]^s a[1]^
  FullSimplify[p - s, ass] a[2]^ FullSimplify[q - s, ass] a[3]^FullSimplify[r -
  s, ass]];
```

The commands below generate useful Euler factors for normalization:

```
InvL[X_, d_, nota_ : a] := Product[(1 - nota[i] X), {i, 1, d}]
InvRSL[X_, d_, nota1_ : a, nota2_ : b] := Product[Product[(1 - nota1[i]*nota2[j]X)
  , {i, 1, d}], {j, 1, d}]
Syma2 = a[1] - a[2];
Symb2 = b[1] - b[2];
Syma3 = (a[1] - a[2]) (a[1] - a[3]) (a[2] - a[3]);
Symb3 = (b[1] - b[2]) (b[1] - b[3]) (b[2] - b[3]);
```

We use `Hec[m]` and `dHec[n]` to denote $\lambda_{\Pi}(p^m)$ and $\lambda_{\tilde{\Pi}}(p^n)$, respectively; `A[1]` and `A[2]` to denote the linear and quadratic elementary symmetric polynomials in `a[]` (similarly for `B[1]`, `B[2]` in `b[]`).

For Π on $\text{PGL}_3(\mathbb{A}_{\mathbb{Q}})$, the notation `Fou[m,n]` refers to $\lambda_{\Pi}(p^m, p^n)$. The commands below express `Fou[m,n]` in `Hec[m]` and `dHec[n]`, and implement the Hecke recurrence relation:

```
UnrRepla = Fou[m_, n_] :> dHec[m]*Hec[n] - dHec[m - 1]*Hec[n - 1];
RamRepla = Fou[m_, n_] :> Hec[m + n]*Hec[m] - Hec[m + n + 1]*Hec[m - 1];
HecRecur[k_] := Hec[k] -> Hec[1]*Hec[k - 1] - A[2]*Hec[k - 2] + \[Omega]*Hec[k - 3];
```

The following commands are used in pattern-matching with the Schur polynomials:

```
Rulpl={a[1] :> Hec[1], a[1]^k_. - a[2]^k_. :> Hec[k- 1], a[1]^(k + n_.)a[2]^r_.
- a[1]^(k + n_.)a[3]^r_. + a[2]^(k + n_.)a[3]^r_. - a[2]^(k + n_.)a[1]^r_. +
a[3]^(k + n_.)a[1]^r_. - a[3]^(k + n_.)a[2]^r_. :> Fou[r-1, k +n-r-1]};

Rulmin = {a[1] :> Hec[1], -a[1]^k_. + a[2]^k_. :> -Hec[k - 1], -a[1]^(k + n_.) a
[2]^r_. + a[1]^(k + n_.) a[3]^r_. - a[2]^(k + n_.) a[3]^r_. + a[2]^(k + n_.)
a[1]^r_. - a[3]^(k + n_.) a[1]^r_. + a[3]^(k + n_.) a[2]^r_. :> -Fou[r - 1, k
+ n - r - 1]};

Char\[Omega_] := a[1]^k_. a[2]^k_. a[3]^k_. :> \[Omega];
Replsch = Fou[m_., n_.] :> SchurPolynomial[{a[1], a[2], a[3]}, {m, n}];
```